Documento técnico

Script Registros

Sistema de Registro y Visualización de Varamientos

PMPUy – Tramo 2

Elaborado por:

Costeros
CONSERVACIÓN URUGUAY

Elaborado por	Germán Leguizamón – Costeros Conservación Uruguay
Revisado por	Germán Leguizamón – Costeros Conservación Uruguay
Versión	1.0
Fecha última actualización	06/04/26

Contenido

1. Objetivo del documento 4
2. Alcance 4
3. Objetivo del script 4
4. Script completo analizado 5
5. Constantes de configuración 7
6. Función principal: runPMPUyBackup() 8
7. Función de recorrido y sincronización: processFolder(folder, pathInGithub) 9
8. Función de exportación: exportGoogleFile(fileId, mimeType) 13
9. Función de subida: uploadToGithub(path, content, message) 16
10. Lógica interna de decisión del script 18
11. Qué hace cada bloque condicional dentro de processFolder() 19
12. Estructura de datos y persistencia de estado 19
13. Operatividad práctica del proceso completo 20
14. Conclusión técnica 20

1. Objetivo del documento

Este documento explica la lógica y las funciones utilizadas para elaborar el Script Registros, quien controla la automatización del Sistema de Registro y Visualización de Varamientos.

2. Alcance

El destino de este documento es para integrantes del grupo Costeros Conservación Uruguay, que quieren acceder a información técnica del desarrollo del script.

3. Objetivo del script

El script implementa un mecanismo de respaldo automatizado que recorre una carpeta raíz de Google Drive, procesa todos sus archivos y subcarpetas, convierte a formatos estándar los archivos nativos de Google cuando corresponde, detecta si hubo cambios reales en el contenido y sincroniza únicamente los elementos modificados con un repositorio de GitHub.

En términos operativos, el sistema busca resolver cuatro necesidades simultáneas: preservar estructura, reducir trabajo manual, evitar subidas redundantes y dejar una copia externa versionada en GitHub.

4. Script completo analizado

```
/**
 * CONFIGURACIÓN
 */
const GITHUB_TOKEN = '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';
const REPO_OWNER = 'germanl8';
const REPO_NAME = 'registros-kml';
const ROOT_FOLDER_ID = '1l5FmAI5I3Qi0R-UHzlBL8B4RQQU80KdU';
const BACKUP_DIR_NAME = 'Respaldo_PMPUy';

/**
 * FUNCIÓN PRINCIPAL
 */
function runPMPUyBackup() {
  const rootFolder = DriveApp.getFolderById(ROOT_FOLDER_ID);
  Logger.log("Iniciando respaldo PMPUy...");

  processFolder(rootFolder, BACKUP_DIR_NAME + "/");

  Logger.log("Respaldo finalizado con éxito.");
}

/**
 * PROCESAR CARPETAS RECURSIVAMENTE
 */
function processFolder(folder, pathInGithub) {
  const files = folder.getFiles();
  const subFolders = folder.getFolders();
  const scriptProperties = PropertiesService.getScriptProperties();

  while (files.hasNext()) {
    const file = files.next();
    const fileId = file.getId();
    const mimeType = file.getMimeType();
    let fileName = file.getName();

    try {
      let blob;
      let exportType = mimeType;

      if (mimeType === MimeType.GOOGLE_SHEETS) {
        blob = exportGoogleFile(fileId, 'application/vnd.openxmlformats-
officedocument.spreadsheetml.sheet');
        fileName += ".xlsx";
        exportType = "xlsx";

      } else if (mimeType === MimeType.GOOGLE_DOCS) {
        blob = exportGoogleFile(fileId, 'application/pdf');
        fileName += ".pdf";
        exportType = "pdf";

      } else if (mimeType === MimeType.GOOGLE_SLIDES) {
        blob = exportGoogleFile(fileId, 'application/pdf');
        fileName += ".pdf";
        exportType = "pdf";

      } else if (mimeType === MimeType.GOOGLE_FORMS) {
        Logger.log("Form no exportable, se omite: " + fileName);
        continue;

      } else if (mimeType.includes('vnd.google-apps')) {
        Logger.log("Tipo Google no soportado, se omite: " + fileName);
        continue;
```

```
      } else {
        blob = file.getBlob();
      }

      const bytes = blob.getBytes();
      const contentHash = Utilities.base64Encode(
        Utilities.computeDigest(
          Utilities.DigestAlgorithm.MD5,
          bytes
        )
      );

      const propertyKey = fileId + "_" + exportType;
      const lastHash = scriptProperties.getProperty(propertyKey);

      const fullPath = pathInGithub + fileName;

      if (lastHash && lastHash === contentHash) {
        Logger.log("Sin cambios: " + fullPath);
        continue;
      }

      const base64Content = Utilities.base64Encode(bytes);
      const response = uploadToGithub(fullPath, base64Content, "Backup PMPUy: " + fileName);

      if (response.getResponseCode() == 200 || response.getResponseCode() == 201) {
        scriptProperties.setProperty(propertyKey, contentHash);
        Logger.log("Sincronizado: " + fullPath);
      } else {
        Logger.log("Error API en " + fileName + ": " + response.getContentText());
      }

    } catch (e) {
      Logger.log("Error procesando " + fileName + ": " + e.message);
    }
  }

  while (subFolders.hasNext()) {
    const subFolder = subFolders.next();
    processFolder(subFolder, pathInGithub + subFolder.getName() + "/");
  }
}

/**
 * EXPORTAR ARCHIVOS GOOGLE (Drive API)
 */
function exportGoogleFile(fileId, mimeType) {
  const url =
`https://www.googleapis.com/drive/v3/files/${fileId}/export?mimeType=${encodeURIComponent(mimeTyp
e)}`;

  const token = ScriptApp.getOAuthToken();

  const response = UrlFetchApp.fetch(url, {
    headers: {
      Authorization: 'Bearer ' + token
    },
    muteHttpExceptions: true
  });

  if (response.getResponseCode() !== 200) {
    throw new Error("Error exportando archivo: " + response.getContentText());
  }

  return response.getBlob();
}
```

```
/**
 * SUBIR A GITHUB
 */
function uploadToGithub(path, content, message) {
  const url = `https://api.github.com/repos/${REPO_OWNER}/${REPO_NAME}/contents/${path}`;

  const headers = {
    "Authorization": "token " + GITHUB_TOKEN,
    "Accept": "application/vnd.github.v3+json"
  };

  let sha = null;

  const checkRes = UrlFetchApp.fetch(url, {
    method: "GET",
    headers: headers,
    muteHttpExceptions: true
  });

  if (checkRes.getResponseCode() === 200) {
    sha = JSON.parse(checkRes.getContentText()).sha;
  }

  const payload = {
    message: message,
    content: content,
    sha: sha
  };

  const options = {
    method: "PUT",
    headers: {
      "Authorization": "token " + GITHUB_TOKEN,
      "Content-Type": "application/json"
    },
    payload: JSON.stringify(payload),
    muteHttpExceptions: true
  };

  return UrlFetchApp.fetch(url, options);
}
```

5. Constantes de configuración

Antes de ejecutar cualquier función, el script define cinco constantes globales. Estas constantes gobiernan la autenticación, el origen de los archivos, el repositorio destino y la carpeta base dentro del repositorio.

Constante	Qué almacena	Qué hace en la práctica
GITHUB_TOKEN	Token personal de GitHub	Autoriza las llamadas a la API de GitHub para crear o actualizar archivos en el repositorio
REPO_OWNER	Usuario propietario del repositorio	Forma parte de la URL de la API y dirige la subida al dueño correcto
REPO_NAME	Nombre del repositorio	Indica cuál repositorio recibirá los respaldos

ROOT_FOLDER_ID	ID de la carpeta raíz en Drive	Define desde dónde empieza el recorrido recursivo
BACKUP_DIR_NAME	Nombre de carpeta base en GitHub	Hace que todo el contenido se copie dentro de una carpeta principal ordenada

Operativamente, estas constantes evitan valores hardcodeados dispersos dentro del resto del script. Todo el comportamiento general queda centralizado en un bloque de configuración.

6. Función principal: runPMPUyBackup()

Firma de la función:

```
function runPMPUyBackup()
```

Código de la función:

```
function runPMPUyBackup() {
  const rootFolder = DriveApp.getFolderById(ROOT_FOLDER_ID);
  Logger.log("Iniciando respaldo PMPUy...");

  processFolder(rootFolder, BACKUP_DIR_NAME + "/");

  Logger.log("Respaldo finalizado con éxito.");
}
```

Propósito operative

Es la puerta de entrada del proceso completo. No recibe parámetros porque toma toda la configuración necesaria desde las constantes globales. Su responsabilidad es iniciar el respaldo desde la carpeta raíz definida en Drive y delegar el trabajo real de recorrido y sincronización a processFolder().

Parámetros y elementos de entrada

Elemento	Tipo	Explicación exacta
No tiene parámetros	-	La función se ejecuta directamente; usa ROOT_FOLDER_ID y BACKUP_DIR_NAME definidos al inicio del script
rootFolder	Objeto Folder	Se obtiene con DriveApp.getFolderById(ROOT_FOLDER_ID) y representa la carpeta origen desde donde comienza el respaldo

BACKUP_DIR_NAME + "/"	Cadena de texto	Genera la ruta base inicial dentro de GitHub; por ejemplo, Respaldo_PMPUy/

Qué hace exactamente:

1. Obtiene la carpeta raíz de Google Drive usando el identificador ROOT_FOLDER_ID.
2. Escribe en el log un mensaje de inicio. Esto sirve para auditoría, diagnóstico y seguimiento de ejecución.
3. Invoca processFolder(rootFolder, BACKUP_DIR_NAME + "/"). En ese instante transfiere el control a la función recursiva encargada de recorrer toda la estructura.
4. Cuando processFolder() termina de recorrer todo el árbol de carpetas y archivos, registra un mensaje final indicando que el proceso concluyó.

Resultado / efecto esperado

El efecto real de esta función no es subir archivos por sí sola, sino iniciar y encapsular la ejecución global del respaldo completo.

Observaciones técnicas

Es recomendable usar esta función como objetivo de un trigger horario o manual.

Toda la robustez del sistema depende de que ROOT_FOLDER_ID apunte a la carpeta correcta y de que processFolder() opere sin interrupciones críticas.

7. Función de recorrido y sincronización: processFolder(folder, pathInGithub)

Firma de la función:

```
function processFolder(folder, pathInGithub)
```

Código de la función:

```
function processFolder(folder, pathInGithub) {
  const files = folder.getFiles();
  const subFolders = folder.getFolders();
  const scriptProperties = PropertiesService.getScriptProperties();

  while (files.hasNext()) {
    const file = files.next();
```

```
    const fileId = file.getId();
    const mimeType = file.getMimeType();
    let fileName = file.getName();

    try {
      let blob;
      let exportType = mimeType;

      if (mimeType === MimeType.GOOGLE_SHEETS) {
        blob = exportGoogleFile(fileId, 'application/vnd.openxmlformats-
officedocument.spreadsheetml.sheet');
        fileName += ".xlsx";
        exportType = "xlsx";

      } else if (mimeType === MimeType.GOOGLE_DOCS) {
        blob = exportGoogleFile(fileId, 'application/pdf');
        fileName += ".pdf";
        exportType = "pdf";

      } else if (mimeType === MimeType.GOOGLE_SLIDES) {
        blob = exportGoogleFile(fileId, 'application/pdf');
        fileName += ".pdf";
        exportType = "pdf";

      } else if (mimeType === MimeType.GOOGLE_FORMS) {
        Logger.log("Form no exportable, se omite: " + fileName);
        continue;

      } else if (mimeType.includes('vnd.google-apps')) {
        Logger.log("Tipo Google no soportado, se omite: " + fileName);
        continue;

      } else {
        blob = file.getBlob();
      }

      const bytes = blob.getBytes();
      const contentHash = Utilities.base64Encode(
        Utilities.computeDigest(
          Utilities.DigestAlgorithm.MD5,
          bytes
        )
      );

      const propertyKey = fileId + "_" + exportType;
      const lastHash = scriptProperties.getProperty(propertyKey);

      const fullPath = pathInGithub + fileName;

      if (lastHash && lastHash === contentHash) {
        Logger.log("Sin cambios: " + fullPath);
        continue;
      }

      const base64Content = Utilities.base64Encode(bytes);
      const response = uploadToGithub(fullPath, base64Content, "Backup PMPUy: " + fileName);

      if (response.getResponseCode() == 200 || response.getResponseCode() == 201) {
        scriptProperties.setProperty(propertyKey, contentHash);
        Logger.log("Sincronizado: " + fullPath);
      } else {
        Logger.log("Error API en " + fileName + ": " + response.getContentText());
      }

    } catch (e) {
      Logger.log("Error procesando " + fileName + ": " + e.message);
    }
  }
```

```
  while (subFolders.hasNext()) {
    const subFolder = subFolders.next();
    processFolder(subFolder, pathInGithub + subFolder.getName() + "/");
  }
}
```

Propósito operative

Es la función central del sistema. Su trabajo es recorrer una carpeta concreta de Drive, revisar todos sus archivos, decidir cómo tratarlos, determinar si cambiaron y sincronizarlos con GitHub. Después de terminar con los archivos de la carpeta actual, entra recursivamente en cada subcarpeta y repite exactamente el mismo proceso.

Parámetros y elementos de entrada

Parámetro o variable	Tipo	Explicación exacta
folder	Folder	Objeto carpeta de Google Drive que se está procesando en ese momento
pathInGithub	String	Ruta acumulada dentro del repositorio donde se guardarán los archivos de la carpeta actual
files	FileIterator	Iterador que recorre los archivos existentes en folder
subFolders	FolderIterator	Iterador que recorre las subcarpetas existentes en folder
scriptProperties	Properties	Almacén persistente del script donde se guarda el último hash de cada archivo exportado o subido
file	File	Archivo individual extraído del iterador files
fileId	String	Identificador único del archivo en Google Drive
mimeType	String	Tipo MIME del archivo; sirve para decidir si se exporta, se omite o se sube tal cual
fileName	String	Nombre base del archivo. Puede modificarse agregando extensión al exportarlo
blob	Blob	Contenido binario final que efectivamente se subirá o comparará
exportType	String	Etiqueta del formato final usado para el hash; evita confundir el mismo archivo si cambia el formato de exportación
bytes	Byte[]	Arreglo de bytes del blob; base para calcular el hash y para codificar el contenido

contentHash	String	Hash MD5 codificado en base64 del contenido final
propertyKey	String	Clave única con formato fileId_exportType; identifica el hash guardado en propiedades
lastHash	String o null	Último hash guardado previamente para ese archivo y formato
fullPath	String	Ruta completa de destino en GitHub, compuesta por pathInGithub + fileName
base64Content	String	Contenido del archivo en base64, formato requerido por la API de GitHub
response	HTTPResponse	Respuesta devuelta por uploadToGithub()
subFolder	Folder	Subcarpeta encontrada dentro de la carpeta actual

Qué hace exactamente:

1. Crea tres objetos fundamentales: un iterador de archivos, un iterador de subcarpetas y el acceso a Script Properties para consultar el historial de hashes.

2. Entra en un bucle while (files.hasNext()) para procesar uno por uno todos los archivos de la carpeta actual.

3. Para cada archivo obtiene su identificador, tipo MIME y nombre.

4. Dentro del bloque try define dos variables de trabajo: blob, donde quedará el contenido final, y exportType, que inicialmente toma el mimeType del archivo.

5. Evalúa el tipo MIME. Si el archivo es Google Sheets, lo exporta a XLSX; si es Google Docs o Google Slides, lo exporta a PDF; si es Google Forms, lo omite porque ese tipo no se exporta con esta lógica; si es otro tipo vnd.google-apps no contemplado, también lo omite; si no es un archivo nativo de Google, toma el blob directo del archivo.

6. Una vez obtenido el blob, transforma su contenido en bytes y calcula un hash MD5. Ese hash representa el contenido real del archivo que se quiere respaldar.

7. Construye propertyKey combinando fileId y exportType. Esto es importante porque el mismo archivo de Drive puede terminar respaldado en un formato exportado diferente al original.

8. Consulta en scriptProperties si existe un hash anterior para esa clave. Si existe y coincide exactamente con el hash actual, interpreta que no hubo cambios reales y por lo tanto omite la subida.

9. Arma la ruta completa de destino en GitHub con pathInGithub + fileName.

10. Si detecta cambios o si no hay hash previo, codifica el archivo en base64 y llama a uploadToGithub().

11. Evalúa el código de respuesta HTTP. Si GitHub responde 200 o 201, guarda el nuevo hash en Script Properties y deja constancia en log de que la sincronización fue exitosa.

12. Si la respuesta no es exitosa, registra el detalle completo devuelto por la API.

13. Si cualquier error ocurre al procesar un archivo concreto, el catch registra el problema y permite que el resto de los archivos continúen procesándose.

14. Después de terminar con los archivos, entra en otro while sobre subFolders. Por cada subcarpeta encontrada, vuelve a llamarse a sí misma agregando el nombre de la subcarpeta al pathInGithub. Esta es la base de la recursividad.

Resultado / efecto esperado

El resultado de esta función es doble: por un lado sincroniza los archivos necesarios de la carpeta actual; por otro, garantiza que todas las subcarpetas queden recorridas y respaldadas manteniendo la estructura original.

Observaciones técnicas

Es la función que materializa la lógica incremental del sistema.

La combinación de recursividad + hash + Script Properties evita recorridos parciales y reenvíos innecesarios.

Cuando un archivo no cambia, el script no vuelve a consumir llamadas PUT contra GitHub para ese archivo.

8. Función de exportación: exportGoogleFile(fileId, mimeType)

Firma de la función:

```
function exportGoogleFile(fileId, mimeType)
```

Código de la función:

```
function exportGoogleFile(fileId, mimeType) {
  const url =
`https://www.googleapis.com/drive/v3/files/${fileId}/export?mimeType=${encodeURIComponent(mimeTyp
e)}`;

  const token = ScriptApp.getOAuthToken();
```

```
  const response = UrlFetchApp.fetch(url, {
    headers: {
      Authorization: 'Bearer ' + token
    },
    muteHttpExceptions: true
  });

  if (response.getResponseCode() !== 200) {
    throw new Error("Error exportando archivo: " + response.getContentText());
  }

  return response.getBlob();
}
```

Propósito operativo

Se encarga exclusivamente de convertir un archivo nativo de Google Drive a un formato estándar descargable. En otras palabras, obtiene una copia exportada del archivo usando la API REST de Google Drive.

Parámetros y elementos de entrada

Parámetro o variable	Tipo	Explicación exacta
fileId	String	ID del archivo de Google Drive que se quiere exportar
mimeType	String	Formato de salida solicitado a la API; por ejemplo PDF o XLSX
url	String	Endpoint de exportación construido dinámicamente con el ID del archivo y el mimeType codificado
token	String	Token OAuth temporal del script, necesario para autenticarse ante Google Drive API
response	HTTPResponse	Respuesta HTTP devuelta por la API de exportación

Qué hace exactamente:

1. Construye la URL de exportación usando el patrón /files/{fileId}/export?mimeType=....
2. Codifica el mimeType con encodeURIComponent() para evitar errores por caracteres especiales en la URL.
3. Obtiene el token OAuth del contexto de ejecución con ScriptApp.getOAuthToken().

4. Hace la llamada HTTP con UrlFetchApp.fetch(), enviando el token como Bearer en el header Authorization.

5. Activa muteHttpExceptions: true para poder leer el cuerpo de la respuesta incluso si la API devuelve error.

6. Verifica el código HTTP de la respuesta. Si no es 200, lanza una excepción detallando el contenido devuelto por la API.

7. Si la exportación fue correcta, devuelve response.getBlob(), es decir, el archivo exportado en binario.

Resultado / efecto esperado

Devuelve un Blob listo para ser convertido en bytes, hasheado y eventualmente subido a GitHub.

Observaciones técnicas

La función no sube nada ni guarda estado; su responsabilidad es únicamente obtener el archivo exportado.

Es una función reusable: cualquier otra parte del sistema podría usarla si necesitara exportar un archivo de Google.

9. Función de subida: uploadToGithub(path, content, message)

Firma de la función:

```
function uploadToGithub(path, content, message)
```

Código de la función:

```
function uploadToGithub(path, content, message) {
  const url = `https://api.github.com/repos/${REPO_OWNER}/${REPO_NAME}/contents/${path}`;

  const headers = {
    "Authorization": "token " + GITHUB_TOKEN,
    "Accept": "application/vnd.github.v3+json"
  };

  let sha = null;

  const checkRes = UrlFetchApp.fetch(url, {
    method: "GET",
    headers: headers,
    muteHttpExceptions: true
  });

  if (checkRes.getResponseCode() === 200) {
    sha = JSON.parse(checkRes.getContentText()).sha;
  }

  const payload = {
    message: message,
    content: content,
    sha: sha
  };

  const options = {
    method: "PUT",
    headers: {
      "Authorization": "token " + GITHUB_TOKEN,
      "Content-Type": "application/json"
    },
    payload: JSON.stringify(payload),
    muteHttpExceptions: true
  };

  return UrlFetchApp.fetch(url, options);
}
```

Propósito operativo

Administra la escritura efectiva del archivo en GitHub. Primero consulta si el archivo ya existe en la ruta indicada; si existe, obtiene su SHA actual. Luego arma un payload y ejecuta una operación PUT para crear o actualizar el contenido.

Parámetros y elementos de entrada

Parámetro o variable	Tipo	Explicación exacta
path	String	Ruta exacta del archivo dentro del repositorio, incluyendo carpetas y nombre final
content	String	Contenido del archivo codificado en base64, exigido por la API de GitHub Contents
message	String	Mensaje de commit que quedará registrado en el historial del repositorio
url	String	Endpoint de Contents API apuntando al archivo específico
headers	Objeto	Headers para autenticación y negociación con GitHub
sha	String o null	SHA del archivo ya existente en GitHub; si no existe, permanece en null
checkRes	HTTPResponse	Respuesta del GET previo usado para comprobar existencia
payload	Objeto	Cuerpo JSON que GitHub necesita para crear o actualizar el archivo
options	Objeto	Opciones de la llamada PUT: método, headers, payload y manejo de errores HTTP

Qué hace exactamente:

1. Construye la URL de la API apuntando a /repos/{owner}/{repo}/contents/{path}.
2. Prepara headers con el token de GitHub y el Accept de la versión de API utilizada.
3. Inicializa sha en null. Esto significa que, por defecto, se intentará crear un archivo nuevo.
4. Realiza una petición GET a la misma URL para averiguar si el archivo ya existe.
5. Si GitHub devuelve código 200, interpreta que el archivo existe y extrae el campo sha del JSON de respuesta.
6. Construye el objeto payload con tres campos: message para el commit, content en base64 y sha. Si sha es null, GitHub lo interpreta como creación; si trae un SHA válido, lo interpreta como actualización de un archivo existente.

7. Prepara las opciones de la petición PUT, incluyendo serialización JSON del payload.

8. Ejecuta UrlFetchApp.fetch(url, options) y devuelve la respuesta sin procesarla internamente.

Resultado / efecto esperado

Devuelve la respuesta HTTP de GitHub para que la función llamadora pueda decidir si la operación fue exitosa o falló.

Observaciones técnicas

La separación entre esta función y processFolder() permite mantener desacoplada la lógica de sincronización respecto de la lógica de comunicación con GitHub.

La inclusión del SHA es obligatoria cuando se actualiza un archivo existente mediante la API Contents de GitHub.

10. Lógica interna de decisión del script

La operatividad real del sistema puede resumirse como una cadena de decisiones. Esta cadena explica por qué el script es incremental y no una simple copia masiva en cada ejecución.

Etapa	Decisión	Consecuencia operativa
1. Obtener archivo	¿Qué tipo de archivo es?	Define si se exporta, se omite o se toma directamente
2. Obtener blob	¿Ya existe contenido exportado/descargado?	Permite calcular hash y preparar la subida
3. Calcular hash	¿El hash actual coincide con el guardado?	Si coincide, se evita la re-subida
4. Preparar destino	¿Cuál es la ruta exacta en GitHub?	Mantiene estructura espejo de carpetas
5. Subir a GitHub	¿El archivo ya existe en el repositorio?	Si existe se actualiza con SHA; si no existe se crea
6. Guardar hash	¿La API respondió 200 o 201?	Se persiste el nuevo estado para futuras ejecuciones
7. Recursividad	¿Existen subcarpetas?	Se procesa todo el árbol sin importar profundidad

11. Qué hace cada bloque condicional dentro de processFolder()

Dado que processFolder() concentra la mayor parte de la operatividad, a continuación se desglosan los condicionales exactos que gobiernan el comportamiento del script.

Condición	Qué detecta	Acción concreta
mimeType === MimeType.GOOGLE_SHEETS	Hoja de cálculo nativa de Google	Se exporta a XLSX y se añade extensión .xlsx al nombre
mimeType === MimeType.GOOGLE_DOCS	Documento nativo de Google	Se exporta a PDF y se añade extensión .pdf
mimeType === MimeType.GOOGLE_SLIDES	Presentación nativa de Google	Se exporta a PDF y se añade extensión .pdf
mimeType === MimeType.GOOGLE_FORMS	Formulario de Google	Se omite por no estar contemplada una exportación utilizable en esta lógica
mimeType.includes('vnd.google-apps')	Otro archivo nativo de Google no soportado	Se omite para evitar errores de exportación no manejados
lastHash && lastHash === contentHash	El contenido actual ya fue respaldado en la última ejecución exitosa	No se sube nuevamente
response.getResponseCode() == 200 || 201	GitHub aceptó crear o actualizar el archivo	Se guarda el nuevo hash en Script Properties

12. Estructura de datos y persistencia de estado

El script no usa una base de datos externa. En su lugar, persiste el estado mínimo necesario usando PropertiesService.getScriptProperties(). El estado guardado es el hash del último contenido sincronizado de cada archivo.

La clave utilizada es fileId + "_" + exportType. Esto es importante porque el identificador del archivo de Drive por sí solo no distingue el formato de salida utilizado en la exportación. Al incluir exportType, el script evita colisiones lógicas.

Componente	Finalidad
fileId	Identifica de forma única el archivo en Drive
exportType	Identifica el formato final respaldado (por ejemplo pdf o xlsx)
propertyKey	Une ambos valores para crear una clave estable y reutilizable
contentHash	Representa el estado actual del contenido respaldado

13. Operatividad práctica del proceso completo

Se ejecuta runPMPUyBackup().

Se localiza la carpeta raíz en Google Drive.

Se recorre la carpeta actual y luego todas sus subcarpetas.

Cada archivo se clasifica por MIME type.

Si es necesario, el archivo se exporta a un formato estándar.

Se calcula un hash del contenido real a respaldar.

Se compara con el hash guardado de la última sincronización.

Solo si hay diferencias, el archivo se codifica en base64 y se envía a GitHub.

GitHub crea o actualiza el archivo y devuelve una respuesta HTTP.

Si la respuesta es correcta, se actualiza el hash persistido.

14. Conclusión técnica

El script está correctamente dividido en funciones con responsabilidades claras: una función de inicio, una función central de recorrido y sincronización, una función especializada en exportación desde Google Drive y una función dedicada a la interacción con GitHub. Su diseño combina recursividad, conversión de formatos, control de cambios por hash y persistencia de estado mínima para construir un respaldo incremental eficiente y trazable.